StutterClear

Implementation guide for the team · Nebius × NVIDIA Global AI Hackathon

Deadline	**Friday 30 October 2026, 10:00 am PT = 10:30 pm IST** (submit on Devpost; feature freeze 27 Oct)
Repository	https://github.com/Pradyuman-aviator/stutterClear
Status	The app works end to end on one laptop: live cleaning, voice notes, real-recording demos, captions, four Nemotron features (built, not yet tested with a real key), hosted mode, Docker deployment files.
What is left	Nebius key, the hosted demo on Nebius AI Cloud, model calibration, 4-6 features, user tests, the video.

Read this first

- StutterClear removes stutters (repeated words, broken-off starts, optionally fillers) from speech **in the speaker's own voice**: live in calls and from recordings. Nothing is re-synthesised; we only cut the original audio.
- Every work package below says **what** to build, **where** in the code, **how**, and **when it is done**. Put your name on the ones you take (the Owner line) and tell the group.
- Priorities: **P0** = we fail the hackathon without it, **P1** = makes us competitive, **P2/P3** = only if time allows.
- Rules that are not negotiable are in section 7 (never tune on the 7 test speakers, never commit keys or model files, tests and the UI build must pass before every commit).
- The full technical description is the repository README; this guide is the to-do list with directions.

1. How we meet the hackathon requirements

Requirement (official rules)	How StutterClear meets it	Status
Runs on Nebius Token Factory or Nebius AI Cloud	Nemotron calls go to Token Factory (stutterclear/nemotron.py). The hosted demo is meant to run on an AI Cloud GPU VM (docs/DEPLOY.md).	Code done; needs the key (WP1) and the VM (WP2)
Uses at least one NVIDIA open model	NVIDIA Nemotron 3 Super (meaning guard, word rescue, context agent, insights) and NVIDIA FastConformer (NeMo streaming ASR, live captions).	Done
Demo URL	Public HTTPS link to the hosted app.	WP2
Video under 3 minutes	Script in docs/SUBMISSION.md.	WP10
Public open-source repository	GitHub, MIT licence	Done
New or significantly updated during the period	First commit 17 Oct 2026.	Done

Judging (equal weight): technological implementation (effective use of Token Factory / AI Cloud and Nemotron), design (a complete product), potential impact, quality of the idea (creative, non-obvious use of Nemotron).

2. Architecture

The browser talks to one FastAPI server. Live audio goes in over a WebSocket and live updates (waveforms, cuts, captions) come back over another. Recordings and voice notes are cleaned on the GPU. Only text ever goes to the cloud (Nemotron, Tavily).

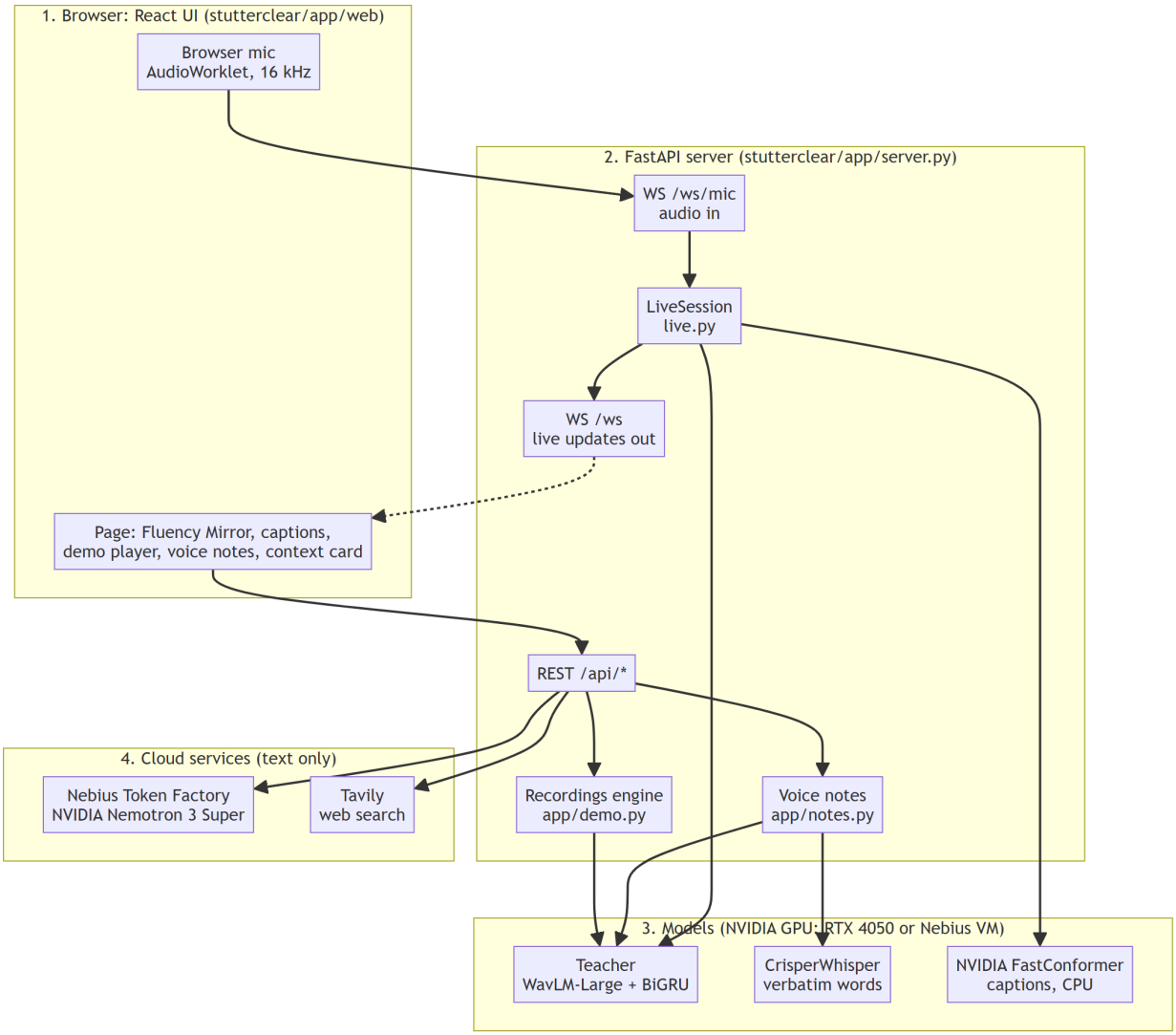


Figure 1. System overview: browser, server, models on the GPU, cloud services.

2.1 The live pipeline

Your voice is played a fraction of a second late. During that delay the big model (the *teacher*) looks at sliding windows of audio and marks stutters; the splicer skips what was marked before the listener hears it. Safety rules: it never plays audio the detector has not judged, it fills gaps with the room's own background noise (never digital silence), and it drops any cut still open after 2.5 s.

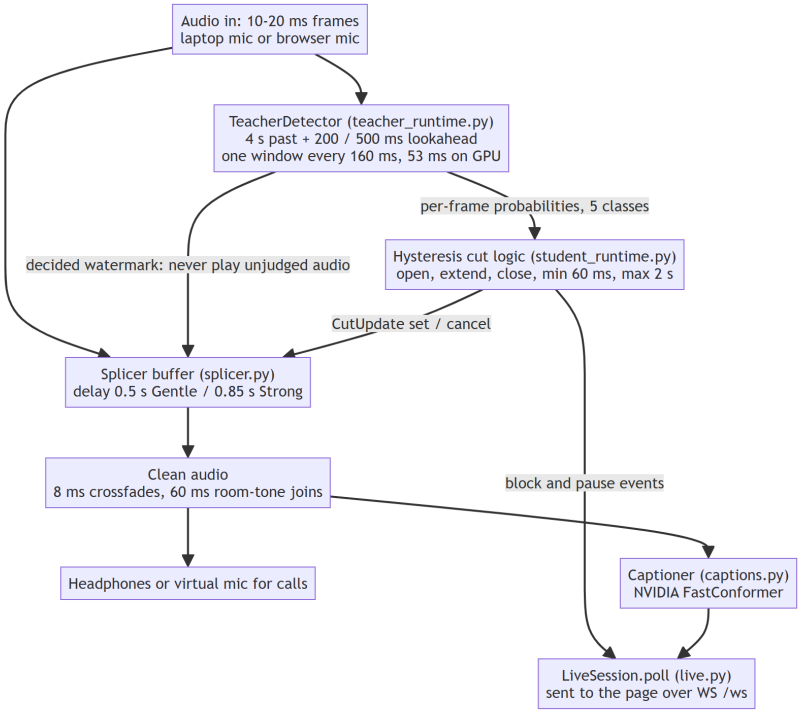


Figure 2. Live pipeline. Files: live.py, teacher_runtime.py, student_runtime.py, splicer.py, captions.py.

2.2 Recordings (voice notes and demo clips)

For recordings we have time to transcribe. CrisperWhisper writes down every word with timings, stutters included; cuts land on word boundaries only, the teacher must agree, and Nemotron's meaning guard keeps repeats the person meant ("I know that that is true").

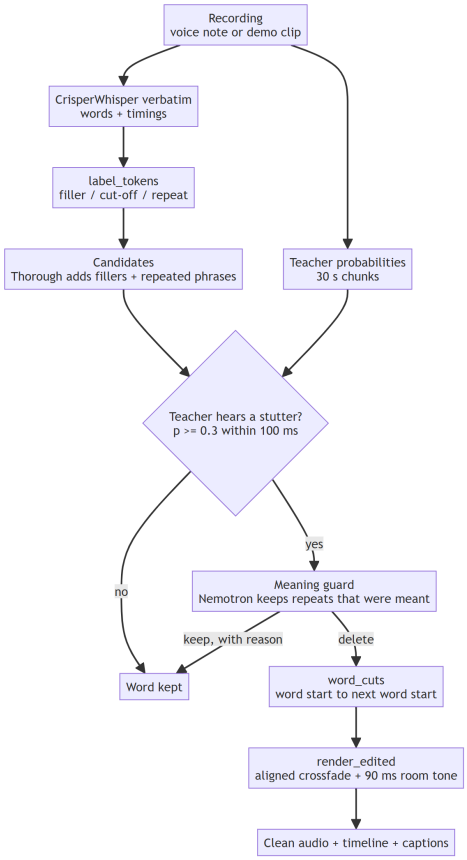


Figure 3. Recording engine. Files: app/demo.py, app/notes.py, silver.py (word_cuts), guard.py, segments.py.

2.3 Hosted deployment

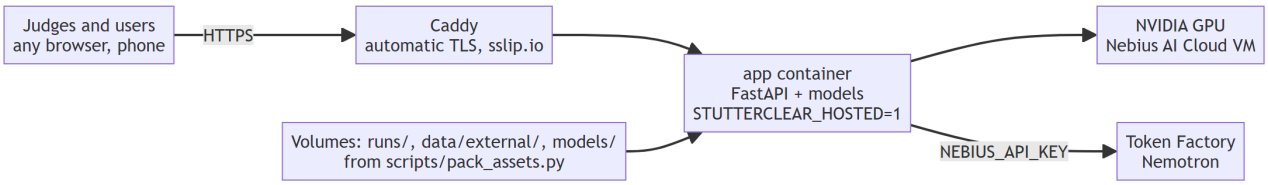


Figure 4. Hosted demo: Caddy (HTTPS) → app container on a Nebius AI Cloud GPU VM. Files: Dockerfile, docker-compose.yml, deploy/Caddyfile, docs/DEPLOY.md.

3. Codebase map

Path	What it does
stutterclear/splicer.py	Real-voice splicer: delay buffer, cuts, crossfades, room tone, catch-up in pauses.
stutterclear/live.py	LiveSession (mic or browser audio), detector choice (make_detector, recommended), poll() for the page.
stutterclear/teacher.py, teacher_runtime.py	Teacher model (WavLM-Large + BiGRU) and its live sliding-window detector.
stutterclear/student.py, student_runtime.py	Small causal student and the online hysteresis cut logic (shared with the teacher).
stutterclear/detector.py	Rule-based fallback detector, CutUpdate and DetectorEvent types.
stutterclear/segments.py	Offline cut and join logic (CutConfig, JoinConfig, render_edited).
stutterclear/silver.py	Labels from verbatim transcripts (label_tokens), word-aligned cuts (word_cuts), scoring.
stutterclear/captions.py	NVIDIA FastConformer streaming captions via sherpa-onnx.
stutterclear/nemotron.py	Token Factory client: chat(), model check and fallback (resolve()), JSON parsing.
stutterclear/guard.py, rescue.py, context_agent.py, insights.py	The Nemotron features: meaning guard (+ typed try-it), word rescue, context agent, insights.
stutterclear/app/server.py	FastAPI: REST /api/*, WebSockets /ws and /ws/mic, hosted-mode limits.
stutterclear/app/demo.py, notes.py	Recording engines (natural / thorough / live) and voice notes.
stutterclear/app/web/src	React UI (Vite). Built into stutterclear/app/static, which is committed.
scripts/	run_app.py, eval_live.py, eval_recordings.py, check_nemotron.py, training and data scripts.
kaggle/	GPU training jobs (silver labels, teacher v2, student v2).
tests/	pytest suite (43 tests). Must pass before every commit.
runs/eval/	Measured results the app reads (live_quality.json) and reports.

4. Set up your machine

Windows 10/11 or Linux, Python 3.11+, Node 20+ (only for UI work). An NVIDIA GPU with ~5 GB free memory for the big model; without one the app runs with the rule-based detector.

```
git clone https://github.com/Pradyuman-aviator/stutterClear.git
cd stutterClear
pip install -r requirements.txt        # install the CUDA build of PyTorch first (pytorch.org)
tar xf stutterclear_assets.tar         # models + demo data, ask Pradyuman (private, see below)
python scripts/run_app.py --port 8766  # http://127.0.0.1:8766
python -m pytest -q                    # all tests must pass

# UI changes: edit stutterclear/app/web/src, then build into stutterclear/app/static
cd stutterclear/app/web && npm install && npm run build
```

Model files and data are **not** in git (size and licences: UCLASS and SEP-28k are research-only). **stutterclear_assets.tar** (made by scripts/pack_assets.py) holds the teacher checkpoint, the measured results, the demo recordings with their caches, and the captions model. Share it only inside the team. Without it: download captions with scripts/download_asr.py and UCLASS with scripts/download_uclass.py.

Environment variable	Purpose
NEBIUS_API_KEY	Enables Nemotron. Never in code, chat, screenshots or git.
TAVILY_API_KEY	Optional web search for the context agent.
NEBIUS_BASE_URL, NEBIUS_MODEL	Override the Token Factory endpoint and model (defaults are fine).
STUTTERCLEAR_HOSTED=1	Public-server mode: browser mic only, per-tab privacy, limits.
STUTTERCLEAR_TEACHER	Path to another teacher checkpoint (to test teacher v2).

5. Work packages

WP	What	Prio	Effort	Needs	Owner
1	Nebius key + test every Nemotron feature	P0	0.5 d	a card	
2	Hosted demo on Nebius AI Cloud (or plan B)	P0	1 d	WP1, VM access	
3	Speech mode + "Say it" button	P1	2 d		
4	Interview practice partner (Nemotron)	P1	1.5-2 d	WP1	
5	Fluency aid in your ear (delayed feedback)	P2	0.5-1 d		
6	Human feedback loop (accept / reject cuts, personal thresholds)	P2	1.5 d		
7	Calibrate teacher v2 on validation speakers	P1	1 d	GPU	
8	Benchmark student v2	P1	0.5 d	Kaggle job done	
9	Download timestamps (CSV / subtitles)	P3	0.5 d		
10	Submission: video, users, Devpost	P0	2 d	WP1, WP2	

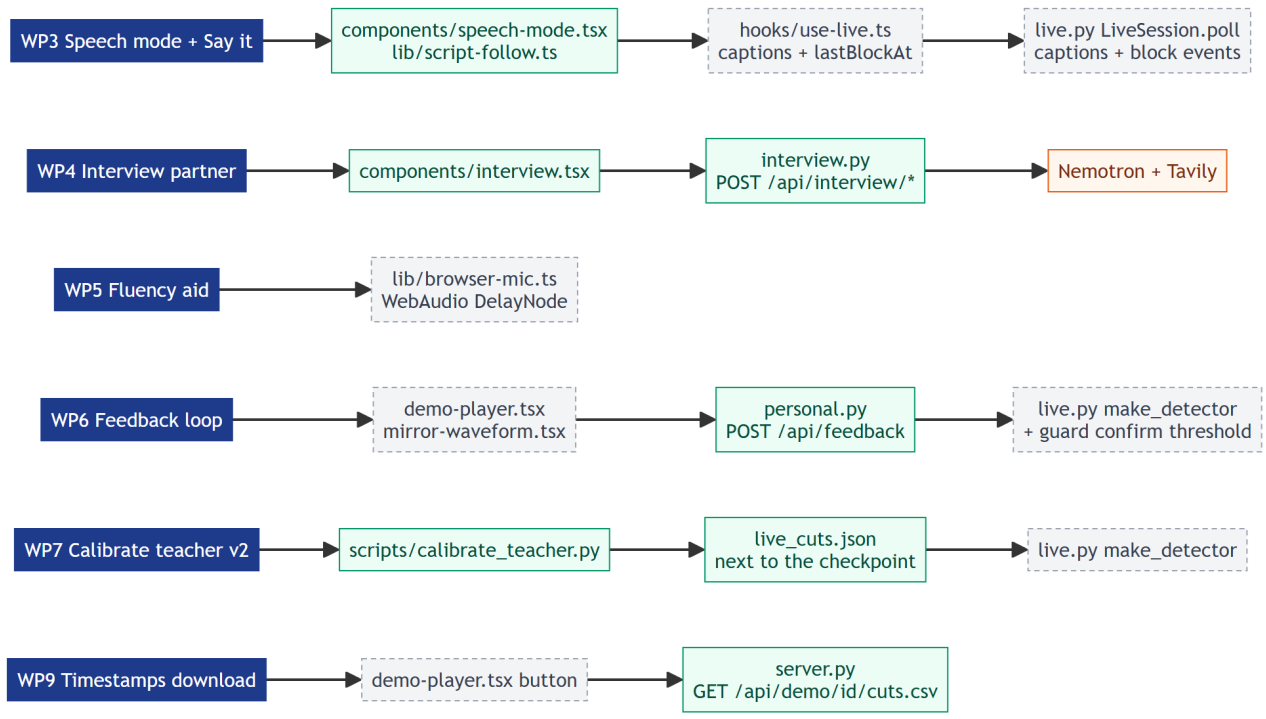


Figure 5. Where each feature plugs in. Green = new file or route, grey dashed = existing code to extend, orange = external service.

WP1 Nebius key and Nemotron check

Priority P0 · **Effort** 0.5 day · **Owner** _____

Goal

Real Nemotron answers in all four features and the meaning-guard try-it, so the demo and the video show them working.

Where in the code

- stutterclear/nemotron.py: client (chat, resolve() picks the closest served model if the name changed).
- scripts/check_nemotron.py: tests chat, word rescue, meaning guard (with a known answer), insights, context agent.

How

1. Sign up at tokenfactory.nebius.com; billing details (Individual, full address); a bank card is required even with promo codes. Indian cards: enable international/online transactions first.
2. Claim credits: the hackathon form with activation code **NEBIUS-DEVPOST-GLOBAL26** ($25) and the Builders Program ($25). Redeem: balance → Top up → With promo code.
3. Create an API key; set it where the app runs: `setx NEBIUS_API_KEY "..."` and open a new terminal.
4. Run `python scripts/check_nemotron.py`; then restart the app (startup renders the demo clips with the guard) and click through every Nemotron feature.

Done when

- check_nemotron.py prints "all good" (the guard keeps "that" in "that that is true" and cuts the repeated "I").
- The demo player shows a "Meaning guard · Nemotron" line on the real recordings; write down what it kept for the video.

Watch out

- If the balance goes negative, Nebius charges the card automatically. Watch Organisation → Usage; the hosted server caps visitors at 60 Nemotron calls a minute.

WP2 Hosted demo on Nebius AI Cloud

Priority P0 · **Effort** 1 day · **Owner** _____

Goal

A public HTTPS link where judges can try live (browser mic), voice notes, demos and Nemotron.

Where in the code

- Dockerfile, docker-compose.yml, deploy/Caddyfile, .env.example, scripts/pack_assets.py, docs/DEPLOY.md (step by step).

How

1. Create a GPU VM (Ubuntu with NVIDIA drivers, ~5 GB GPU memory is enough, ports 22/80/443 open); check `docker run --rm --gpus all nvidia/cuda:12.1.1-base-ubuntu22.04 nvidia-smi`.
2. Clone the repo on the VM; copy stutterclear_assets.tar privately (scp) and extract it.
3. cp .env.example .env; set DOMAIN (the VM IP with dashes + .sslip.io), NEBIUS_API_KEY, TAVILY_API_KEY.
4. `docker compose up -d --build`; watch `docker compose logs -f app` (first start downloads WavLM and CrisperWhisper).
5. Plan B if the VM is not ready by 26 Oct: the laptop with `STUTTERCLEAR_HOSTED=1` behind `cloudflared tunnel --url http://localhost:8766` (in DEPLOY.md).

Done when

- The https:// link works in Chrome, Edge, Firefox and on a phone: demo clips play, a browser-mic live session cleans speech, a voice note comes back cleaned, Nemotron features answer.
- Two tabs at once: the second sees none of the first one's captions or audio (hosted privacy).

Watch out

- Never make the assets public (licences). Keep .env on the server only.

WP3 Speech mode + "Say it" button

Priority P1 · **Effort** 2 days · **Owner** _____

Goal

For a prepared talk: the person pastes the speech; the app follows along with live captions; when they get stuck (a block), the next word appears big on screen; pressing a key can speak it. Nothing is ever spoken without that press.

Where in the code

- New components/speech-mode.tsx: script box, big "next word" display, Say it button + hotkey; mount it in App.tsx (e.g. a tab next to the Context card).
- New lib/script-follow.ts: alignment of what was said to the script.
- hooks/use-live.ts (exists): the caption store (lines + partial) and data.current.lastBlockAt (set when the server sends a block event). live.py poll() already sends both; no backend change is needed for phase 1.

How

1. Split the script into normalised words (lower case, no punctuation).
2. On every caption update, take the last ~8 spoken words and find the best match in a window of ±30 words around the current position (fuzzy word match, e.g. similarity ≥ 0.7); move the pointer to just after it.
3. Show the next 1-3 words big when lastBlockAt changes, or when no new caption word arrives for ~700 ms in the middle of a sentence.
4. Say it (phase 1): `speechSynthesis.speak(new SpeechSynthesisUtterance(word))`, only on the button or a hotkey (Space / PageDown, which presentation clickers send).
5. Phase 2 (stretch, for calls): add a Splicer method that inserts a short audio clip at the output position and a server route that synthesises the word, so the word reaches the call too.

Done when

- Reading a 1-minute script aloud, the pointer stays within 2 words of the speaker.
- A mid-sentence pause shows the next word within 1 s; Say it speaks it; nothing is spoken automatically.
- Works without a script too (falls back to Nemotron word rescue suggestions).

Watch out

- Speaking for the person is the ethical line: only on their explicit press, and say so in the UI.

WP4 Interview practice partner

Priority P1 · **Effort** 1.5-2 days · **Owner** _____

Goal

Nemotron plays an interviewer for a role and company, asks questions out loud, the user answers by voice; after each answer: the clean playback and three supportive insights. A strong, creative use of Nemotron.

Where in the code

- New stutterclear/interview.py: next_question(role, card, history) via nemotron.chat(..., think=False).
- stutterclear/context_agent.py (exists): fill(situation) researches the company once (Tavily if set).
- app/server.py: POST /api/interview/start {role, company} → {card, question}; POST /api/interview/next {history} → {question}. Call nemotron_budget() in both (hosted credit cap).
- New components/interview.tsx: question card (read aloud with speechSynthesis), Answer = the existing BrowserMic live session (lib/browser-mic.ts), then Playback + InsightsPanel.

How

1. System prompt: a realistic, friendly interviewer; one question at a time; follow up on the last answer; never comment on the stutter.
2. Keep the last ~6 question/answer pairs as history (answers from the captions).
3. After each answer, call /api/insights with the transcript and the cut counts from the session summary.
4. Add a unit test with a stand-in model (see tests/test_server.py: test_nemotron_features_with_a_stand_in_model).

Done when

- A 5-question mock interview runs end to end with the real key; each answer has clean playback and insights.
- Without a key the panel says it needs NEBIUS_API_KEY (HTTP 503), like the other features.

WP5 Fluency aid in your ear

Priority P2 · **Effort** 0.5-1 day · **Owner** _____

Goal

Optional delayed auditory feedback: the speaker hears their own voice 50-200 ms late in headphones (a known technique many people who stutter find helpful), while the listener still gets the cleaned voice.

Where in the code

- lib/browser-mic.ts: after the mic source, add a branch source → DelayNode(0.05-0.2 s) → GainNode → ctx.destination.
- App.tsx Setup card: on/off switch + delay slider (stored in localStorage like other settings).
- Device mode (later): live.py, a second sounddevice OutputStream fed with the delayed raw input.

How

1. Build the WebAudio branch; it must not change what is sent to the server.
2. Headphones only: show a warning, because speakers feed back into the mic.

Done when

- With the switch on you hear yourself delayed by the slider value; the cleaned output is unchanged.

Watch out

- Wording: "some people find it helps". No cure or treatment claims anywhere.

WP6 Human feedback loop

Priority P2 · **Effort** 1.5 days · **Owner** _____

Goal

Users accept or reject each cut and tap missed stutters; the app learns personal thresholds and shows an agreement score. Auto mode is offered once agreement is ≥ 95% over the last 50 decisions.

Where in the code

- components/demo-player.tsx: a list of the cuts (time, kind, word) with Accept / Reject buttons for voice notes.
- components/mirror-waveform.tsx: tap on the waveform = "missed one here".
- New stutterclear/personal.py + POST /api/feedback: store decisions (local runs/feedback/*.jsonl; on the hosted server keep them in the browser's localStorage instead) and update per-class threshold multipliers.
- live.py make_detector and guard.guarded_cuts: apply the multipliers.

How

1. Multiplier per class starts at 1.0; a rejected cut of a class raises it by 5%, a missed stutter lowers it by 5%; clamp to 0.5-1.5.
2. Agreement = accepted / (accepted + rejected + missed) over a rolling window of 50.
3. Show the score next to the mode switch; offer Auto at ≥ 95%.

Done when

- Rejecting a class's cuts several times visibly reduces that class's cuts on the next note.
- Unit test for the update rule and the clamp.

WP7 Calibrate teacher v2

Priority P1 · **Effort** 1 day · **Owner** _____

Goal

Teacher v2 finds more stutters than v1 but, with v1's thresholds, also cuts more real words live (7.5% vs 3.1%). Tune its thresholds on validation speakers, test once, and switch only if it wins.

Where in the code

- New scripts/calibrate_teacher.py; live.py make_detector (read live_cuts.json next to the checkpoint); teacher_runtime.py TEACHER_CKPT; scripts/eval_live.py.

How

1. Validation data: the 49 UCLASS recordings whose speaker has uclass_split(...) == "val" (stutterclear/train_data.py). Their word timings are in runs/kaggle_silver/silver/words_*.jsonl; write them as <wav>.crisper.json next to each wav.
2. Cache live-style probabilities with teacher_lookahead.streamed_probs (lookahead 200 and 500 ms, hop 160 ms).
3. Sweep per-class thresholds (start from Kaggle's validation-tuned v2 values 0.36 / 0.64 / 0.41 / 0.25 / 0.65), off_ratio and merge_gap with the online cut logic run offline (fast, CPU). Objective: removed − 3 × lost.
4. Write the best settings to runs/kaggle_teacher_v2/teacher_v2/live_cuts.json.

```
STUTTERCLEAR_TEACHER=runs/kaggle_teacher_v2/teacher_v2/best.pt \
  python scripts/eval_live.py --detectors teacher --out runs/eval/live_teacher_v2.json
```

6. If v2 beats v1 (33.1% / 3.10% Gentle, 65.2% / 5.68% Strong) by removed − 3 × lost: make v2 the default, regenerate the demo caches (.teacher.npy), update runs/eval/live_quality.json and the README.

Done when

- A one-time test on the 7 test speakers decides; the result goes in the README either way.

Watch out

- Never tune on the 7 test speakers (data/external/uclass_eval). That would make every number meaningless.

WP8 Benchmark student v2

Priority P1 · **Effort** 0.5 day · **Owner** _____

Goal

Student v2 (5.2M parameters) runs on CPUs. It should replace the rule-based detector on laptops without a GPU if it measures better.

Where in the code

- Downloaded to runs/kaggle_student_v2 when the Kaggle job ends; live.py DEFAULT_STUDENT; scripts/eval_live.py.

How

```
python scripts/eval_live.py --detectors student \
  --student runs/kaggle_student_v2/student_v2/student.pt \
  --out runs/eval/live_quality.json --merge
```

2. The app recommends a detector automatically from live_quality.json (removed − 3 × lost).

Done when

- Numbers recorded; the app recommends the student on CPU-only machines only if it beats rule-based (7.6% removed, 1.25% lost).

WP9 Download timestamps

Priority P3 · **Effort** 0.5 day · **Owner** _____

Goal

Export the detected stutters of a voice note (start, end, kind, words) as CSV and as subtitles; useful for speech therapists.

Where in the code

- app/server.py: GET /api/demo/{id}/cuts.csv and cuts.srt from demo.render(...)["timeline"]["cuts"].
- components/demo-player.tsx: a Download timestamps button next to Download clean.

How

1. CSV columns: start_s, end_s, kind, words. SRT: one cue per cut.
2. Add a test in tests/test_server.py.

Done when

- Downloading from a cleaned voice note gives a file that opens in Excel and in a video player.

WP10 Submission

Priority P0 · **Effort** 2 days · **Owner** _____

Goal

Everything the judges see: the demo URL, the video, the Devpost page, real users.

Where in the code

- docs/SUBMISSION.md (Devpost text, criteria map, video script, testing steps for judges), docs/img/.

How

1. 3-5 people who stutter try it (with consent); collect short quotes.
2. Record the video (under 3 minutes, headphones on): a real recording before/after, the meaning-guard moment, live with captions, word rescue, the numbers.
3. Devpost: the Representative creates the project, invites teammates (Manage team), adds the demo URL, repo, video, gallery images; agree the prize split in writing first.

Done when

- Submitted before 30 Oct, 10:30 pm IST, with every teammate on the project.

6. Timeline

Dates	Work	Notes
18-19 Oct	WP1 key + check, WP8 student benchmark	Nemotron must work before anything else
20-21 Oct	WP3 Speech mode + Say it	
22-23 Oct	WP4 Interview partner, WP7 calibration	In parallel (different people)
24 Oct	WP5 fluency aid, WP9 timestamps	Small ones
24-25 Oct	WP6 feedback loop	
25-26 Oct	WP2 deploy + full test on the public URL	Plan B decision on 26 Oct
27 Oct	Feature freeze; users try it	Only bug fixes after this
28 Oct	Record the video	
29-30 Oct	Devpost text, screenshots, final checks	Submit by 30 Oct 10:30 pm IST

7. Engineering rules

- **Never tune anything on the 7 test speakers** (data/external/uclass_eval). Tune on validation speakers, test once.
- **Never commit** keys, .env, model checkpoints, UCLASS or SEP-28k audio, or the assets tar.
- Before every commit: `python -m pytest -q` passes; for UI changes `npm run build` passes (it type-checks) and the built stutterclear/app/static is committed with the source.
- Small commits, one logical change each, pushed right away; describe what changed and why.
- Product promises: the listener only hears the speaker's real voice; nothing is spoken for them unless they press Say it; no cure or treatment claims; text (never audio) goes to Nemotron, and only when a feature is used.
- Hosted mode must stay private per tab and within the limits (5-minute live turns, 4 waiting voice notes, 60 Nemotron calls a minute).

8. Numbers to quote (7 held-out UCLASS speakers)

What	Result
Live, teacher v1, Gentle (0.5 s delay)	33.1% of stutters removed, 3.10% of words lost
Live, teacher v1, Strong (0.85 s delay)	65.2% removed, 5.68% lost
Live, rule-based (no GPU)	7.6% removed, 1.25% lost
Recordings, Natural / Thorough	50.1% / 80.7% removed, no real word cut
Teacher v2, official SEP-28k-E test	macro F1 0.618; AUC 0.98 word repetitions, 0.96 fillers
Meaning kept on the demo clips	word error rate 0.07-0.15 against the original
Speed	one 5 s window in 53 ms on a laptop RTX 4050 (a decision every 160 ms)

Links: repository · hackathon resources · Token Factory docs · Nebius Discord